

-
- 一、两阶段提交&&checkpoint实现逻辑
- 二、FlinkKafkaProducer逻辑
 - 1.initializeState
 - 2.snapshotState()
 - 3.preCommit() //预提交
 - 4.beginTransaction() //开启下一个事物
 - 5.commit //提交
- 三、rtlp下游kafka频繁报错原因分析

一、两阶段提交&&checkpoint实现逻辑

1. CheckpointCoordinator, 检查点协调器, 负责协调flink算子的state分布式快照, 在触发快照的时候, CheckpointCoordinator会向算子中注入Barrier消息。每当需要做checkpoint时, JobManager就在数据流中打入一个屏障 (barrier), 作为检查点的界限。
2. 算子收到barrier之后, 开始执行实现的snapshotState()方法
3. 在进行checkpoint的同时, 让下游sink开始执行preCommit, 此时没真正提交事物
4. 于此同时对sink做一个 beginTransaction, 开启下一个事务
5. 完成Checkpoint后, 所有的Subtask都会收到JobManager发来的NotifyCheckpointComplete消息, 在该方法中调用了最后真正的commit()方法完成事务型的提交, 完成两阶段提交的过程

二、FlinkKafkaProducer逻辑

1.initializeState

```
public void initializeState(FunctionInitializationContext context) throws Exception {
    .....
    this.nextTransactionalIdHintState = context.getOperatorStateStore().getUnionListState(NEXT_TRANSACTIONAL_ID_HINT_DESCRIPTOR);
    if (context.getOperatorStateStore().getRegisteredStateNames().contains(NEXT_TRANSACTIONAL_ID_HINT_DESCRIPTOR)) {
        this.migrateNextTransactionalIdHintState(context);
    }
    .....
    // taskName是该算子的名称+OperatorUniqueID(通过拓扑结构得出, 所以算子拓扑结果相同, 改值固定)+IndexOfThisSubtask (区分并行度中的每个算子)
    this.transactionalIdsGenerator = new TransactionalIdsGenerator(taskName + "-" + ((StreamingRuntimeContext) this).getRuntimeContext().IndexOfThisSubtask());
    if (this.semantic != FlinkKafkaProducer.Semantic.EXACTLY_ONCE) {
        this.nextTransactionalIdHint = null;
    } else {
        ArrayList<FlinkKafkaProducer.NextTransactionalIdHint> transactionalIdHints = Lists.newArrayList((Iterable) this.nextTransactionalIdHintState);
        if (transactionalIdHints.size() > 1) {
            throw new IllegalStateException("There should be at most one next transactional id hint written by the first subtask");
        }
        if (transactionalIdHints.size() == 0) {
            //这意味着这是: 此应用程序的第一次执行
            this.nextTransactionalIdHint = new FlinkKafkaProducer.NextTransactionalIdHint(0, 0L);
            //上一次执行在第一个检查点完成之前失败, 我们必须中止以前的所有事务, 通过generateIdsToAbort遍历得到transactionalIds, 依次中止之前的事务
            this.abortTransactions(this.transactionalIdsGenerator.generateIdsToAbort());
        } else {
            this.nextTransactionalIdHint = (FlinkKafkaProducer.NextTransactionalIdHint) transactionalIdHints.get(0);
        }
    }
}

super.initializeState(context);
}
```

注意: 从Flink角度来看, 可能同时存在多个checkpoint, 此时意味着同一个sink operator也会同时存在多个, 而kafka的transaction id同一时刻只能有一个, 为了避免出现事务冲突, flink引入了kafkaProducersPoolSize的参数进行配合, 默认值为5, 理论上能够同时对同一个sink operator能够支持5个checkpoint同时进行。下面便是该队列的声明语句, 它是一个双向并发阻塞队列, 大小为kafkaProducersPoolSize默认5, 同时在使用完一个transaction id, 会重新放回该队列中, 实现重复使用。

```
this.availableTransactionalIds = new LinkedBlockingDeque();
```

2.snapshotState()

```
public void snapshotState(FunctionSnapshotContext context) throws Exception {
    super.snapshotState(context);
    this.nextTransactionalIdHintState.clear();
    //为了避免重复, 只有第一个subtask跟踪下一个事务id提示。否则所有的subtask将写入完全相同的信息。
    if (this.getRuntimeContext().getIndexOfThisSubtask() == 0 && this.semantic == FlinkKafkaProducer.Semantic.EXACTLY_ONCE) {
        Preconditions.checkState(this.nextTransactionalIdHint != null, "nextTransactionalIdHint must be set for EXACTLY_ONCE");
        long nextFreeTransactionalId = this.nextTransactionalIdHint.nextFreeTransactionalId;
        //在使用Flink的自动缩容和扩容时 (Reactive Mode), 需要更具当前的并行度调整TransactionalIds的数量。
        if (this.getRuntimeContext().getNumberOfParallelSubtasks() > this.nextTransactionalIdHint.lastParallelism) {
            nextFreeTransactionalId += (long) (this.getRuntimeContext().getNumberOfParallelSubtasks() * this.kafkaProducersPoolSize);
        }
    }
}
```

```
//将本次获得的TransactionalId传入，作为本算子sink的事务Id
this.nextTransactionalIdHintState.add(new FlinkKafkaProducer.NextTransactionalIdHint(this.getRuntimeContext().getNumberOf
```

3.preCommit() //预提交

```
protected void preCommit(FlinkKafkaProducer.KafkaTransactionState transaction) throws FlinkKafkaException {
    switch(this.semantic) {
        case EXACTLY_ONCE:
        case AT_LEAST_ONCE:
            // 保证目前的缓存的record数据全部写入send中
            this.flush(transaction);
        case NONE:
            this.checkErroneous();
            return;
        default:
            throw new UnsupportedOperationException("Not implemented semantic");
    }
}
```

该动作由flink客户端触发，实际流程有kafka客户端进行实际运行，flink客户端只负责调用回调函数进行判断，具体见第三节总结

4.beginTransaction() //开启下一个事物

```
protected FlinkKafkaProducer.KafkaTransactionState beginTransaction() throws FlinkKafkaException {
    switch(this.semantic) {
        case EXACTLY_ONCE:
            FlinkKafkaInternalProducer<byte[], byte[]> producer = this.createTransactionalProducer();
            //将TransactionalId赋值给KafkaProducer，脱离flink由kafka完成数据的真正写入到下游kafka partition中。
            producer.beginTransaction();
            return new FlinkKafkaProducer.KafkaTransactionState(producer.getTransactionId(), producer);
        case AT_LEAST_ONCE:
        case NONE:
            FlinkKafkaProducer.KafkaTransactionState currentTransaction = (FlinkKafkaProducer.KafkaTransactionState)this.currentTransaction;
            if (currentTransaction != null && currentTransaction.producer != null) {
                return new FlinkKafkaProducer.KafkaTransactionState(currentTransaction.producer);
            }

            return new FlinkKafkaProducer.KafkaTransactionState(this.initNonTransactionalProducer(true));
        default:
            throw new UnsupportedOperationException("Not implemented semantic");
    }
}
```

5.commit //提交

```
protected void commit(FlinkKafkaProducer.KafkaTransactionState transaction) {
    if (transaction.isTransactional()) {
        try {
            transaction.producer.commitTransaction();
        } finally {
            //将旧的TransactionalId循环利用，重新放入队列
            this.recycleTransactionalProducer(transaction.producer);
        }
    }
}
```

6.abort //中止

```
protected void abort(FlinkKafkaProducer.KafkaTransactionState transaction) {
    if (transaction.isTransactional()) {
        //结束此次事务提交
        transaction.producer.abortTransaction();
        //将旧的TransactionalId循环利用，重新放入队列
        this.recycleTransactionalProducer(transaction.producer);
    }
}
```

```
}

```

下图为kafka producer的全流程



上图是一个 KafkaProducer 在发送消息的整个流程，主要关键节点如下：

- 1. ProducerInterceptors 对消息进行拦截
- 2. Serializer 对消息的 key 和 value 进行序列化
- 3. Partitioner 为消息选择合适的 Partition
- 4. RecordAccumulator 收集消息，实现批量发送
- 5. Sender 从 RecordAccumulator 获取消息
- 6. 构造 ClientRequest
- 7. 将 ClientRequest 交给 NetworkClient，准备发送
- 8. NetworkClient 将请求放入 KafkaChannel 的缓存
- 9. 执行网络 I/O，发送请求
- 10. 收到响应，调用 ClientRequest 的回调函数
- 11. 调用 RecordBatch 的回调函数，最终调用每个消息上注册的回调函数

对于目前Flink，RecordAccumulator也就是前5步是发生在flink client中，后续的全部流程是发生在kafka client中，flink会在第4步声明回调函数进行对后续流程的监听。

下面是此次报错的主要error信息：

```
[ ] - Sink: Sink_To_show:dsp.charge.6.cpmshow (7/30)#16 (3327cf64e11d9aebb510f51592bce2ab) switched from RUNNING to FAILED with failure-
cause: org.apache.flink.streaming.connectors.kafka.FlinkKafkaException: Failed to send data to Kafka: Expiring 21 record(s) for
dsp.charge.6.cpmshow-27:120010 ms has passed since batch creation
51   at org.apache.flink.streaming.connectors.kafka.FlinkKafkaProducer.checkErroneous(FlinkKafkaProducer.java:1392)
52   at org.apache.flink.streaming.connectors.kafka.FlinkKafkaProducer.invoke(FlinkKafkaProducer.java:850)
53   at org.apache.flink.streaming.connectors.kafka.FlinkKafkaProducer.invoke(FlinkKafkaProducer.java:99)
54   at org.apache.flink.streaming.api.functions.sink.TwoPhaseCommitSinkFunction.invoke(TwoPhaseCommitSinkFunction.java:223)
55   at org.apache.flink.streaming.api.operators.StreamSink.processElement(StreamSink.java:54)
56   at org.apache.flink.streaming.runtime.tasks.OneInputStreamTask$StreamTaskNetworkOutput.emitRecord(OneInputStreamTask.java:205)
57   at org.apache.flink.streaming.runtime.io.AbstractStreamTaskNetworkInput.processElement(AbstractStreamTaskNetworkInput.java:134)
58   at org.apache.flink.streaming.runtime.io.AbstractStreamTaskNetworkInput.emitNext(AbstractStreamTaskNetworkInput.java:105)
59   at org.apache.flink.streaming.runtime.io.StreamOneInputProcessor.processInput(StreamOneInputProcessor.java:66)
60   at org.apache.flink.streaming.runtime.tasks.StreamTask.processInput(StreamTask.java:423)
```

该错误信息通过分析，主要是flink kafka设置的下面回调函数中报错，主要就是sender线程数据传输过程中的网络IO问题，原因是由于一个batch中数据较多。该问题主要是kafka client 端的问题，于Flink客户端并没有关联。

```
this.callback = new Callback() {
    public void onCompletion(RecordMetadata metadata, Exception exception) {
        if (exception != null && FlinkKafkaProducer.this.asyncException == null) {
            FlinkKafkaProducer.this.asyncException = exception;
        }

        FlinkKafkaProducer.this.acknowledgeMessage();
    }
};
```

	sink成功之后source执行之前失败	sink成功之前失败
hdfs恢复	exactly-once	丢数据
consumer的offsets恢复	重复消费	exactly-once

无标签